

Part. 1

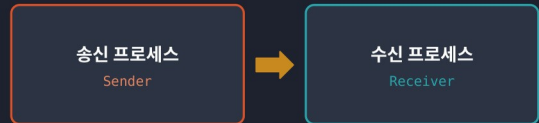
IPC

Inter-Process Communication

프로세스 간 통신

발표자 Janghwan Kim (Harley)

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증



- 01 전역변수 + 뮤텝스 / 세마포어
- 02 공유메모리 + 링버퍼 (메시지 큐)
- 03 TCP / IP 소켓

발표 대본

표지 · 약 30 초

안녕하십니까 . Part.1 과제인 IPC, 프로세스 간 통신 발표를 맡은 Janghwan Kim 입니다 .

이번 과제의 목표는 " 멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능을 구현하고 검증하는 것 " 입니다 .

발표는 크게 두 덩어리입니다 . 앞부분에서는 프로세스와 스레드가 무엇이고 어떤 방법으로 데이터를 주고받는지 , 그리고 IPC 수단들의 특징과 장단점을 정리하겠습니다 . 뒷부분에서는 그 개념들이 실제 코드에서 어떻게 쓰였는지 ToDo 세 개를 실행 결과와 함께 보여드리겠습니다 .

목표

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증

01

쓰레드 / 프로세스 개념 및 통신 방법

개념

리눅스 환경에서 프로그램 · 프로세스 · 쓰레드가 무엇이고, 서로 어떻게 데이터를 주고받는지 정리한다.

02

IPC 개념 및 뮤텝스 / 세마포어

개념

UDP · TCP/IP · MessageQ · 링버퍼 · 공유메모리의 특징과 사용 상황, 장 / 단점, 그리고 이를 안전하게 묶어주는 동기화 수단.

03

데이터 송 / 수신 기능 구현 및 코드 정리

구현

송신 · 수신 쓰레드를 구성해 1 부터 100 까지 0.1 초 간격으로 주고받는다.
ToDo#1 쓰레드 간 · ToDo#2 프로세스 간 · ToDo#3 TCP / IP

오늘 발표는 1 → 2 → 3 순서로, 개념을 먼저 정리하고 그 개념이 코드 어디에 쓰였는지 이어서 보여드립니다.

발표 대본

목표 · 약 1 분

먼저 오늘 다룰 세 가지입니다.

첫 번째, 리눅스 환경에서의 쓰레드와 프로세스 개념, 그리고 통신 방법입니다. 프로그램 · 프로세스 · 쓰레드가 무엇이 다른지, 그 차이 때문에 통신 방법이 왜 갈리는지를 정리합니다.

두 번째, IPC 개념과 동기화입니다. UDP, TCP/IP, 메시지 큐, 링버퍼, 공유메모리를 각각 특징 · 어떤 상황에서 쓰는지 · 장단점으로 나눠 보고, 이들을 안전하게 묶어 주는 뮤텝스와 세마포어를 다룹니다.

세 번째, 실제 구현입니다. 송신 쓰레드와 수신 쓰레드를 구성해서 1 부터 100 까지 0.1 초 간격으로 주고받는 기능을 ToDo 세 가지 방식으로 구현했고, 실행 결과까지 보여드리겠습니다.

순서는 개념을 먼저 정리하고, 그 개념이 코드 어디에 쓰였는지 이어서 보여드리겠습니다.

01

쓰레드 / 프로세스 개념 및 통신 방법

리눅스 환경 기준

1-1

프로그램 · 프로세스 · 쓰레드

무엇이 다르고, 왜 그 차이 때문에 통신 방법이 갈리는가

1-2

통신 방법

쓰레드 간 · 프로세스 간에 쓸 수 있는 수단과 선택 기준

발표 대본

섹션 1 · 약 20 초

첫 번째 주제입니다. 리눅스 환경 기준으로 쓰레드와 프로세스 개념, 그리고 통신 방법을 보겠습니다.

두 가지로 나눠서, 먼저 1-1 에서 프로그램 · 프로세스 · 쓰레드가 무엇이 다른지 정리하고, 1-2 에서 그 차이에 따라 쓸 수 있는 통신 방법이 어떻게 갈리는지 보겠습니다.

1-1 프로그램 · 프로세스 · 쓰레드

무엇을 공유하고 무엇을 따로 갖는지가 통신 방법을 결정한다

1 프로그램 Program

디스크에 저장된 실행 파일 . 아직 실행되지 않은 명령어와 데이터의 묶음이다 .

- 정적 — 메모리도 자원도 할당되지 않은 상태

2 프로세스 Process

실행 중인 프로그램 . 커널이 독립된 가상 주소공간과 자원 (PID · Process ID, 파일 디스크립터) 을 붙여준 단위다 .

- 서로의 메모리를 볼 수 없다 → 그래서 IPC 가 필요하다

3 쓰레드 Thread

프로세스 안의 실행 흐름 . Code · Data · Heap 은 공유하고 Stack 과 레지스터 (PC : Program Counter · SP : Stack Pointer) 만 따로 갖는다 .

- 전역변수로 바로 주고받는다 → 대신 동기화가 필수다

리눅스에서는 프로세스도 쓰레드도 커널 입장에서 task_struct 하나 . clone() 에 무엇을 공유할지만 다르게 준다 .

프로세스 A (독립된 가상 주소공간)

쓰레드끼리 공유하는 영역

Code

Data (전역변수)

Heap

파일 디스크립터

쓰레드 1 (송신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

쓰레드 2 (수신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

전역변수를 통해 복사 없이 바로 전달 · 단, 동시 접근은 뮤텍스로 보호



프로세스 B

주소공간이 다르다 → 직접 접근 불가

발표 대본

1-1 프로그램 · 프로세스 · 쓰레드 · 약 2 분

세 가지를 한 줄씩 구분하면 이렇습니다 .

프로그램은 디스크에 저장된 실행 파일입니다 . 아직 실행되지 않은 명령어와 데이터의 묶음이고 , 메모리도 자원도 할당되지 않은 정적인 상태입니다 .

프로세스는 그 프로그램이 실행된 것입니다 . 커널이 독립된 가상 주소공간과 PID, 파일 디스크립터 같은 자원을 붙여준 단위입니다 . 여기서 중요한 건 , 프로세스끼리는 서로의 메모리를 볼 수 없다는 점입니다 . 바로 이것 때문에 IPC 라는 별도의 수단이 필요합니다 .

쓰레드는 프로세스 안의 실행 흐름입니다 . 오른쪽 그림을 보시면 , Code · Data · Heap · 파일 디스크립터는 쓰레드끼리 공유하고 , 스택과 레지스터만 쓰레드마다 따로 갖습니다 . 그래서 쓰레드끼리는 전역변수 하나로 바로 데이터를 넘길 수 있습니다 . 대신 두 쓰레드가 같은 변수를 동시에 건드리면 값이 깨지기 때문에 동기화가 반드시 필요합니다 .

한 가지 덧붙이면 , 리눅스에서는 프로세스와 쓰레드가 커널 입장에서 사실 같은 구조체입니다 . 둘 다 task_struct 하나이고 , clone() 을 호출할 때 주소공간을 공유할지 말지만 다르게 주는 것뿐입니다 . 그래서 리눅스에서는 쓰레드 생성 비용이 프로세스보다 훨씬 가볍습니다 .

1-2 통신 방법

주소공간을 공유하느냐, 그리고 얼마나 멀리 보내느냐가 쓸 수 있는 수단을 정한다

① 같은 프로세스 안

쓰레드 ↔ 쓰레드 · 주소공간을 공유한다

전역변수 · 공유 자료구조

+ 뮤텝스 · 세마포어 · 조건변수

주소만 넘기고 데이터는 옮기지 않는다

- 커널도 안 거치고 복사도 없어 가장 빠르다
- 동기화를 빠뜨리면 데이터가 깨지거나 데드락이 난다
- 한 쓰레드가 죽으면 프로세스 전체가 같이 죽는다

lock / unlock · sem_wait / sem_post

ToDo#1 ipcThread.c

② 같은 PC, 다른 프로세스

프로세스 ↔ 프로세스 · 커널 통로를 거친다

메시지 큐 · 공유메모리

+ 뮤텝스 · 세마포어 (직접 구현)

참고 : 파이프 · FIFO · 시그널

- 커널이 메시지 경계와 순서를 지켜 준다
- 공유메모리는 복사가 없고, 메시지 큐는 두 번 복사한다
- 이름만 같으면 기동 순서는 상관없다

msgsnd / msgrcv · shmget · mmap

ToDo#2 ipcMsgQ.c

③ 다른 PC · 다른 프로그램

장비 ↔ 장비 · 네트워크를 건넌다

소켓 (TCP / UDP)

+ 연결 · 유실 · 순서를 직접 처리

IP 와 포트만 알면 붙는다

- 유일하게 다른 장비까지 확장된다
- 헤더와 왕복 지연이 붙어 가장 느리다
- 상대가 리눅스여도 같은 방식으로 붙는다

socket / connect · send / recv

ToDo#3 ipcSocket.c

고르는 기준은 하나 — 얼마나 멀리 보내야 하는가

가까울수록 빠르다. 그만큼 커널이 해 주던 경계와 순서, 동기화가 내 코드로 넘어온다.

본 과제는 이 세 칸을 그대로 ToDo#1 · ToDo#2 · ToDo#3 으로 구현했다

발표 대본

1-2 통신 방법 · 약 1 분 30 초

통신 방법은 결국 " 얼마나 멀리 보내야 하는가 " 하나로 갈립니다. 왼쪽부터 오른쪽으로 한 칸씩 멀어집니다.

① 같은 프로세스 안이면 주소공간을 공유하니까 전역변수나 공유 자료구조를 그대로 씁니다. 메모리 주소만 넘기면 되니 복사도 없고 커널도 거치지 않아 가장 빠릅니다. 대신 동기화를 빠뜨리면 데이터가 깨지거나 데드락이 나고, 한 쓰레드가 죽으면 프로세스 전체가 같이 죽습니다. ToDo#1 이 이 칸입니다.

② 같은 PC 의 다른 프로세스면 주소공간이 분리돼 있어 반드시 커널이 만든 통로를 거칩니다. 메시지 큐는 커널이 메시지 경계와 순서를 지켜 주고, 공유메모리는 같은 물리 메모리를 양쪽에 붙여 놓아서 옮겨 담을 일이 없습니다. 파이프와 시그널도 같은 칸에 있지만 이번 과제에서는 쓰지 않았습니다. ToDo#2 가 이 칸입니다.

③ 다른 PC 나 다른 프로그램이면 소켓입니다. 이 중 유일하게 장비를 넘어가고, 대신 연결과 유실, 순서를 직접 처리해야 해서 가장 느립니다. ToDo#3 이 이 칸입니다.

한 줄로 정리하면, 가까울수록 빠르지만 커널이 대신 해 주던 경계와 순서, 동기화가 그만큼 제 코드로 넘어옵니다.

02

IPC 개념 및 뮤텍스 / 세마포어

각 수단의 특징 · 어떤 상황에서 쓰는지 · 장 / 단점

2-1

UDP · TCP / IP

네트워크를 건너는 통신 · 신뢰성이나 속도나

2-2

MessageQ · 링버퍼 · 공유메모리

같은 PC 안에서 데이터를 옮기는 세 가지 도구

2-3

뮤텍스 · 세마포어

공유 자원을 안전하게 만들어 주는 동기화 수단

발표 대본

섹션 2 · 약 20 초

두 번째 주제입니다. IPC 수단들의 개념과 동기화를 보겠습니다.

세 부분으로 나눠서, 2-1 에서 네트워크를 건너는 UDP 와 TCP/IP 를, 2-2 에서 같은 PC 안에서 데이터를 옮기는 메시지 큐 · 링버퍼 · 공유메모리를, 2-3 에서 이들을 안전하게 만들어 주는 뮤텍스와 세마포어를 보겠습니다.

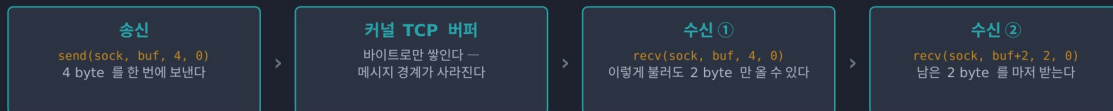
각각 특징, 어떤 상황에서 쓰는지, 장단점 순서로 정리했습니다.

2-1 UDP · TCP / IP

둘 다 IP 위에 올라간다. 차이는 하나 — 신뢰성을 커널이 책임지느냐, 앱이 책임지느냐

구분	TCP / IP Transmission Control Protocol / Internet Protocol	UDP User Datagram Protocol
연결	3-way handshake 로 맺고 시작한다	없음 — 주소만 알면 바로 보낸다
신뢰성	커널이 보장 (재전송 · 순서 · 중복)	보장 없음 — 앱이 직접 처리
데이터 단위	바이트 스트림 — 경계가 없다	데이터그램 — 보낸 크기 그대로
통신 상태	1 : 1 만 가능	1 : N 브로드캐스트 · 멀티캐스트
헤더 · 지연	20 byte · 재전송으로 지연이 된다	8 byte · 가볍고 지연이 낮다
쓰는 곳	파일 전송 · 명령 / 제어 · 로그 수집	센서 · 음성 / 영상 스트리밍
과제 적용	ToDo#3 의 데이터 송 / 수신	TCP Sync 라인체크에만 사용

send 한 번이 recv 한 번으로 오지 않는다



→ 요청한 크기를 다 채울 때까지 반복 수신한다 · f_SocketRecvTCP_IPv4 가 그 루프다

발표 대본

2-1 UDP · TCP/IP · 약 1 분 30 초

둘 다 IP 위에 올라갑니다. 차이는 사실상 하나입니다. 신뢰성을 커널이 책임지느냐, 애플리케이션이 책임지느냐. 표를 위에서부터 한 줄씩 보시면 됩니다. TCP 는 3-way handshake 로 연결을 맺고 시작하고, UDP 는 연결 절차 없이 주소만 알면 바로 보냅니다. 유실이 나면 TCP 는 커널이 재전송하고 순서까지 맞춰 주지만, UDP 는 그 처리를 애플리케이션이 떠안습니다. 그 대가로 TCP 는 헤더가 20 바이트에 재전송 때문에 지연이 튀고, UDP 는 헤더 8 바이트에 지연이 낮습니다.

그래서 쓰는 곳이 갈립니다. 한 건도 빠지면 안 되는 파일 전송이나 명령 제어는 TCP, 최신 값만 의미 있는 센서나 영상 · 음성 스트리밍은 UDP 입니다. 이번 과제에서는 ToDo#3 의 데이터 송수신에 TCP 를 쓰고, UDP 는 TCP Sync 라인체크에만 씁니다.

아래가 구현할 때 실제로 걸린 부분입니다. TCP 는 바이트 스트림이라 메시지 경계가 없습니다. 송신이 4 바이트를 한 번에 보내도 커널 버퍼에는 그냥 바이트로 쌓이고, 수신은 2 바이트만 받을 수 있습니다. 그래서 남은 만큼 다시 받아 요청한 크기를 다 채울 때까지 반복해야 하고, f_SocketRecvTCP_IPv4 가 정확히 그 루프로 되어 있습니다.

예상 질문

3-way handshake 는 왜 3 번인가 — 방향마다 「번호 알림 + 확인」이 필요한데 가운데 SYN+ACK 가 두 역할을 겸해 4 번이 3 번으로 줄었다. 2 번이면 서버가 자기 시작 번호를 상대가 받았는지 알 수 없다.

커널이 신뢰성을 어떻게 보장하나 — 시퀀스 번호로 순서 · 중복을 가리고 ACK 로 확인하며, 안 오면 재전송한다 (중복 ACK 3 회면 즉시). 단 보장 범위는 「상대 커널 버퍼 도착」까지로, 상대 앱이 처리했는지는 별개다.

「스트림에 경계가 없다」가 무슨 뜻인가 — send 를 몇 번 나눠 불렀는지가 사라진다. 그래서 길이 앞에 붙이기 · 구분자 · 고정 길이 중 하나로 앱이 프레이밍해야 한다. 본 과제는 int 4 바이트 고정 길이 방식이다.

TCP 는 왜 1 : 1 인가 — 연결을 (양쪽 IP · 포트) 네 값으로 식별하고 시퀀스 · 윈도우 · 타이머를 그 연결마다 들고 있어서다. 서버가 클라이언트를 여러 받는 것은 연결을 여러 개 만드는 것이다.

헤더가 20 byte 대 8 byte 인 이유 — 늘어난 12 바이트가 시퀀스 번호 · ACK 번호 · 윈도우 크기다. int 4 바이트를 보내면 IP+TCP 헤더가 40 바이트라 전송량의 91% 가 헤더가 된다.

재전송이 왜 지연을 튀게 하나 — RTO 를 채워야 시작하고 실패하면 2 배씩 늘어난다 (200→400→800ms). 게다가 순서 보장 때문에 앞의 하나가 빠지면 뒤에 이미 도착한 것도 같이 대기한다 (HOL).

2-2 메시지 큐 Message Queue

커널이 메시지 단위로 보관해 주는 큐. 보내는 쪽과 받는 쪽이 동시에 떠 있지 않아도 된다

메시지 큐 — 커널이 중간에서 보관한다



★ Windows 에는 System V 메시지 큐가 없다
→ ToDo#2 에서 공유메모리 + 링버퍼로 직접 구현했다

LINUX API (Application Programming Interface)

```
msgget(key, IPC_CREAT) · msgsnd(qid, &msg, size, 0)
msgrcv(qid, &msg, size, mtype, 0) · msgctl(IPC_RMID)
POSIX (Portable Operating System Interface) : mq_open / mq_send
/ mq_receive (우선순위 지원)
```

특징

- 커널 공간에 큐가 만들어지고, 프로세스들은 키 (key) 로 같은 큐를 찾아 붙는다
- 바이트가 아니라 메시지 단위라서 보낸 크기 그대로 받는다
- mtype(메시지 타입) 으로 원하는 종류만 골라 받을 수 있다
- 큐가 비면 받는 쪽이, 가득 차면 보내는 쪽이 커널에서 대기한다

어떤 상황에서 쓰는가

- 명령이나 이벤트처럼 한 건씩 주고받는 데이터
- 보내는 쪽과 받는 쪽 속도가 달라 중간에 쌓아둬야 할 때
- 어느 쪽이 먼저 떠도 상관없어야 할 때

장점

- 커널이 대기까지 해 줘서 따로 락이 필요 없다
- 메시지 경계와 순서가 보장된다
- 상대가 아직 안 떠 있어도 미리 넣어둘 수 있다
- 원하는 타입만 골라서 받을 수 있다

단점

- 유저와 커널 사이를 두 번 복사해 대용량에 불리하다
- 큐에 담기는 양이 정해져 있다 (기본 16KB)
- 지우지 않으면 커널에 계속 남는다
- 같은 PC 안에서만 동작한다

발표 대본

2-2 ① 메시지 큐 · 약 2 분

메시지 큐는 커널이 관리하는 " 메시지 단위 " 의 큐입니다 .

그림처럼 큐 자체가 커널 공간에 만들어지고 , 프로세스들은 키를 가지고 같은 큐를 찾아 붙습니다 . 프로세스 A 가 msgsnd 로 넣고 , 프로세스 B 가 msgrcv 로 꺼냅니다 .

특징은 네 가지입니다 . 바이트가 아니라 메시지 단위라서 보낸 크기 그대로 받습니다 . mtype 이라는 필드로 원하는 종류만 골라 받을 수도 있습니다 . 그리고 큐가 비면 받는 쪽이 , 가득 차면 보내는 쪽이 커널에서 알아서 대기합니다 . 동기화가 이미 들어 있다는 뜻입니다 .

그래서 명령이나 이벤트처럼 " 건 " 단위로 오가는 데이터 , 송수신 속도가 달라 완충이 필요한 경우 , 그리고 두 프로세스의 기동 순서를 맞추기 어려울 때 씁니다 . 커널에 남아 있으니 상대가 아직 안 떠 있어도 미리 넣어 둘 수 있습니다 .

단점은 유저와 커널 사이 복사가 두 번 일어나서 대용량에 불리하고 , 큐 용량이 기본 16KB 정도로 제한된다는 점입니다 .

그리고 아래 표시한 것이 이번 과제의 핵심입니다 . Windows 에는 System V 메시지 큐가 아예 없습니다 . 그래서 ToDo#2 에서는 공유메모리와 링버퍼로 이 동작을 직접 만들었습니다 . 그 이야기를 하려고 다음 두 장에서 링버퍼와 공유메모리를 먼저 보겠습니다 .

예상 질문

msgget 의 key 는 어떻게 정하나 — ftok() 로 파일 경로와 프로젝트 ID 를 조합해 만든다 . 양쪽이 같은 값을 만들어야 같은 큐에 붙는다 .
마지막 인자 0 은 무엇인가 — msgflg 다 . IPC_NOWAIT 를 주면 대기하지 않고 즉시 실패로 돌아온다 . 블로킹이나 논블로킹이나를 여기서 고른다 .

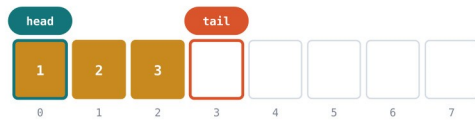
mtype 은 어떻게 쓰나 — msgrcv 에 0 이면 아무거나 , 양수면 그 타입만 , 음수면 절댓값 이하 중 가장 작은 타입을 가져온다 .

msgctl 은 삭제만 하나 — IPC_STAT(상태 조회) · IPC_SET(권한 변경) · IPC_RMID(삭제) 세 가지 명령을 받는다 .

2-2 링버퍼 Ring Buffer

고정 크기 배열을 원형으로 돌려 쓰는 자료구조. IPC 수단이 아니라 데이터를 쌓아 두는 방법이다

고정 크기 배열 8 칸을 원형처럼 돌려 쓴다



끝에 닿으면 $(tail + 1) \% N$ 으로 0 번 슬롯으로 되돌아간다

head — 읽는 위치
수신 쪽이 여기에 꺼낸다

tail — 쓰는 위치
송신 쪽이 여기에 넣는다

```
ring[tail] = msg; tail = (tail+1) % N; count++;
msg = ring[head]; head = (head+1) % N; count--;
```

본 과제 적용

쓰레드 간 8 슬롯 전역 링버퍼 (lpcThread.c)
프로세스 간 64 슬롯 링버퍼를 공유메모리 위에 (lpcMsgQ.c)

특징

- 고정 크기 배열 + 읽는 위치 (head) / 쓰는 위치 (tail) 두 개의 인덱스
- 끝에 닿으면 나머지 연산으로 처음으로 되돌아간다 → $tail = (tail + 1) \% N$
- 개수가 늘어도 넣고 빼는 시간이 그대로다 ($O(1)$), 실행 중 동적 할당도 없다
- 비었는지 찼는지 구별하려면 개수를 세거나 한 칸을 비워 둔다

어떤 상황에서 쓰는가

- 넣는 쪽과 꺼내는 쪽 속도가 다를 때 사이에 두고
- 시리얼 · 네트워크 수신 버퍼, 오디오 · 센서 스트림, 로그 버퍼
- 실행 중에 메모리를 할당하면 안 되는 실시간 코드

장점

- 넣기 / 빼기 시간이 개수와 무관하게 일정 ($O(1)$)
- 메모리 고정 — 단편화도 할당 지연도 없다
- 연속 메모리라 캐시에 유리하다
- 공유메모리 위에 그대로 올릴 수 있다

단점

- 크기가 고정이라 가득 차면 기다리거나 버려야 한다
- head · tail 갱신 자체가 임계구역이다
- 동기화를 직접 붙여야 한다
- 가변 길이 데이터는 담기 번거롭다

발표 대본

2-2 ② 링버퍼 · 약 1 분 30 초

링버퍼는 앞의 두 개와 성격이 조금 다릅니다. 그 자체가 IPC 수단이 아니라, 데이터를 담은 자료구조입니다.

구조는 단순합니다. 고정 크기 배열에 읽는 위치 head 와 쓰는 위치 tail 두 개의 인덱스를 둡니다. 끝에 닿으면 나머지 연산으로 처음으로 돌아갑니다. 아래 코드처럼 넣을 때는 tail, 꺼낼 때는 head 를 하나 늘리고 N 으로 나눈 나머지를 취하는 게 전부입니다.

넣고 빼는 데 $O(1)$ 이고 실행 중에 동적 할당이 전혀 없습니다. 그래서 넣는 쪽과 꺼내는 쪽 속도가 다를 때 사이에 두고 쓰고, 시리얼이나 네트워크 수신 버퍼, 오디오 · 센서 스트림, 그리고 할당 지연이 허용되지 않는 실시간 코드에 씁니다.

단점은 크기가 고정이라 가득 차면 대기하거나 버려야 하고, head 와 tail 을 갱신하는 것 자체가 임계구역이라 동기화를 직접 붙여야 한다는 점입니다.

이번 과제에서는 쓰레드 간에는 8 슬롯 전역 링버퍼를, 프로세스 간에는 64 슬롯 링버퍼를 공유메모리 위에 올려서 썼습니다.

2-2 공유메모리 Shared Memory

두 프로세스가 각자의 주소공간에 같은 물리 메모리를 붙인다. 붙고 나면 그냥 포인터로 읽고 쓴다

같은 물리 메모리를 두 주소공간에 매핑



물리 메모리 (같은 페이지)
양쪽이 이 메모리를 직접 읽고 쓴다 · 옮겨 담지 않는다
※ 순서를 지켜 주는 장치는 여기에 없다

공유메모리 + 뮅스 / 세마포어 는 항상 한 세트다

API — ① 메모리를 만든다 ② 내 주소공간에 붙인다
Linux ① shmget / shm_open ② shmat / mmap
Windows ① CreateFileMapping ② MapViewOfFile
해제 : shmctl / shmctl UnmapViewOfFile / CloseHandle

특징

- 붙일 때만 커널을 거치고, 그 뒤 읽고 쓸 때는 시스템 콜이 없다
- 데이터를 옮겨 담지 않으니 IPC 수단 중 가장 빠르다
- 붙이면 내 주소공간의 주소가 돌아오는데 프로세스마다 다르다 → 포인터 대신 오프셋을 쓴다
- 컴파일러가 끼워 넣는 빈 공간 (패딩) 까지 양쪽이 똑같아야 한다 (#pragma pack)

어떤 상황에서 쓰는가

- 영상 프레임처럼 크고 자주 오가는 데이터
- 한 번 오갈 때 지연이 짧아야 하는 처리
- 여러 프로세스가 같은 데이터를 동시에 읽어야 할 때

장점

- 복사도 시스템 콜도 없어서 가장 빠르다
- 주고받는 크기에 제한이 거의 없다
- 자료구조를 그대로 올릴 수 있다
- 한 번 붙으면 접근 비용이 일정하다

단점

- 동기화 수단이 없어서 직접 붙여야 한다
- 한쪽 실수가 상대 메모리까지 깨뜨린다
- 포인터를 저장하면 안 된다 (주소가 다름)
- 누가 아직 쓰는지 물라 정리 시점을 잡기 어렵다

발표 대본

2-2 ③ 공유메모리 · 약 1 분 30 초

공유메모리는 두 프로세스가 각자의 주소공간에 같은 물리 메모리를 붙여 놓고 쓰는 방식입니다.

그림처럼 프로세스 A 와 B 가 각각 다른 가상 주소로 접근하지만, 실제로는 같은 물리 페이지를 보고 있습니다. 커널은 붙여 주는 일까지만 하고, 그 뒤 읽고 쓸 때는 시스템 콜이 전혀 개입하지 않습니다. 데이터를 옮겨 담지 않으니 IPC 중에서 가장 빠릅니다.

쓰는 순서는 두 단계입니다. 먼저 이름 붙인 메모리 덩어리를 만들고 — 리눅스는 shmget, Windows 는 CreateFileMapping 입니다 — 그 다음 그것을 내 주소공간에 붙입니다. 리눅스는 shmat 이나 mmap, Windows 는 MapViewOfFile 이고, 이 붙이는 함수가 돌려주는 값이 제 주소공간에서 쓸 포인터입니다.

그래서 영상 프레임이나 대형 버퍼처럼 크고 자주 오가는 데이터, 지연이 중요한 실시간 처리에 씁니다.

주의할 점이 두 가지 있습니다. 첫째, 매핑되는 가상 주소가 프로세스마다 다르기 때문에 포인터를 그대로 저장하면 안 됩니다. 반드시 오프셋이나 인덱스를 써야 합니다. 저희 구현에서 head 와 tail 을 인덱스로 둔 이유가 그것입니다. 둘째, 양쪽이 구조체 정렬과 패딩을 똑같이 맞춰야 해서 #pragma pack 으로 고정했습니다.

그리고 가장 중요한 단점은, 공유메모리에는 순서를 지켜 주는 장치가 전혀 없다는 것입니다. 그래서 공유메모리와 뮅스 · 세마포어는 항상 한 세트로 씁니다.

예상 질문

MapViewOfFile 이 정확히 무엇을 하나 — 커널이 들고 있는 메모리 덩어리를 내 프로세스 주소공간에 붙이고 그 시작 주소를 포인터로 돌려준다. 리눅스의 mmap · shmat 에 해당한다.

돌려받은 포인터에 물리 주소가 들어 있나 — 아니다. 내 프로세스의 가상 주소다. 물리 주소는 커널과 MMU 만 다루고 유저 코드는 볼 수 없다.

그런데 왜 포인터를 저장하면 안 되나 — 그 값이 프로세스마다 다른 가상 주소라서다. 상대가 읽어 역참조하면 엉뚱한 곳을 가리킨다. 그래서 head · tail 을 인덱스로 뒀다.

왜 이름이 FileMapping 인가 — 원래 파일을 메모리에 매핑하는 기능이라서다. 첫 인자에 INVALID_HANDLE_VALUE 를 주면 파일 없이 순수 메모리로 쓴다. 본 과제가 그 방식이다.

2-2

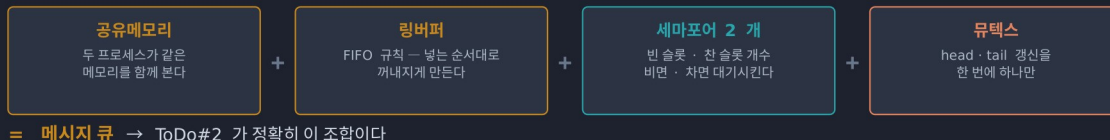
세 가지 비교 — 그리고 조합

메시지 큐는 커널이 주는 기능, 링버퍼는 직접 짜는 자료구조, 공유메모리는 그 링버퍼를 올릴 메모리다

구분	메시지 큐	링버퍼	공유메모리
무엇인가	커널이 관리하는 IPC 객체	자료구조 (IPC 수단 아님)	커널이 연결해 주는 메모리
데이터 단위	메시지 1 개 — 넣은 만큼 나온다	슬롯 1 칸 — 크기 고정	바이트 — 경계는 내가 만든다
복사 횟수	2 회 — 내 변수 → 커널 → 상대	1 회 — 내 변수 → 슬롯	0 회 — 옮겨 담는 일이 없다
동기화	커널이 내장	직접 구현	직접 구현 (필수)
속도	보통	빠름	가장 빠름
용량	커널 메모리라 한도가 있다 (16KB)	고정 — 배열 크기가 곧 용량	물리 메모리가 허용하는 만큼
대표 용도	명령 · 이벤트 전달	들어오는 데이터 임시 보관	대용량 데이터 공유

메시지 큐가 없으면 — 이 넷으로 직접 만든다

세마포어 · 뮤텍스 = 표 동기화 행의 직접 구현 — 다음 장 2-3 에서 설명



발표 대본

2-2 ④ 비교와 조합 · 약 1 분 30 초

셋을 한 표로 정리하면 이렇습니다. 메시지 큐는 커널이 관리하는 IPC 객체이고, 링버퍼는 자료구조이고, 공유메모리는 커널이 연결해 주는 메모리입니다. 성격이 애초에 다릅니다.

가운데 세 줄만 짚겠습니다. 데이터 단위는 넣은 것이 어떤 모양으로 나오느냐입니다. 메시지 큐는 100 바이트를 넣으면 100 바이트가 통째로 나오지만, 공유메모리는 바이트 덩어리라 경계를 제가 만들어야 합니다. 복사 횟수는 데이터가 중간에 몇 번 옮겨지느냐인데, 프로세스는 서로의 메모리를 직접 못 만지니까 메시지 큐는 커널을 거쳐 두 번이고, 공유메모리는 같은 메모리를 보고 있어서 옮겨 담는 일이 없습니다. 속도 차이가 여기서 납니다.

용량은 메시지 큐만 한도가 있는데, 데이터가 커널 메모리에 쌓이기 때문입니다. 커널 메모리는 시스템 전체가 나눠 쓰는 자원이라 리눅스는 큐 하나에 기본 16KB 를 걸어 둡니다. 동기화도 메시지 큐만 커널이 해 줍니다.

아래를 보시면, 셋 중 메시지 큐만 커널이 완성해서 주는 것이고 나머지 둘은 재료입니다. 있으면 그냥 쓰고, 없으면 재료로 직접 만들 수 있습니다. 공유메모리로 두 프로세스가 같은 메모리를 보게 하고, 그 위에 링버퍼를 올려 넣은 순서대로 꺼내지게 하고, 여기에 동기화를 붙입니다. 비었을 때와 찼을 때 대기시키는 세마포어 두 개, head 와 tail 을 보호하는 뮤텍스 하나입니다. 이 둘이 표의 동기화 행에 직접 구현이라고 적힌 그것이고, 바로 다음 2-3 에서 다룹니다.

ToDo#2 가 정확히 이 조합입니다. Windows 에 메시지 큐가 없어서 이렇게 만들었고, 뒤에서 코드로 보여드리겠습니다.

예상 질문

경계 보존이 뭔가 — 100 바이트를 넣으면 100 바이트가 통째로 하나 나온다. 쪼개지지 않는다.

공유메모리는 왜 경계가 없나 — 커널이 그 안을 안 본다. ToDo#2 는 고정 크기 슬롯으로 직접 만들었다.

복사 2 회는 어디를 세나 — 내 변수 → 커널 큐 → 상대 변수. 주소공간이 분리돼 커널을 꼭 거친다.

0 회가 안 쓴다는 뜻인가 — 쓰기는 한다. 쓰는 자리가 곧 상대가 읽는 자리라 옮겨 담기가 없다.

왜 메시지 큐만 한도가 있나 — 데이터가 커널 메모리에 쌓여서다. 리눅스 기본 16KB, 차면 msgsnd 가 대기한다.

메시지 큐를 안 쓰는 건가 — 있으면 그냥 쓴다. Windows 에 그게 없어서 ToDo#2 만 직접 만든 것이다.

왜 여기서 세마포어 · 뮤텍스가 나오나 — 표의 동기화 직접 구현 칸이 이 둘이다. 다음 장 2-3 이 이 둘을 다룬다.

2-3 뮉텍스 Mutex (MUTual EXclusion)

상호배제 — 임계구역에 한 번에 하나만 들어가게 하는 자물쇠

임계구역 — 한 번에 한 명만



임계구역 Critical Section

공유 데이터를 고치는 코드 — 동시에 들어오면 깨진다

MUTEX 잠금

기본 형태 — 실제 함수 이름은 아래 API

lock(m); /* 임계구역 : 공유 데이터 갱신 */ unlock(m);

API — 같은 프로세스 안이나, 프로세스 사이나로 걸린다

Linux pthread_mutex_init → lock / unlock → destroy

Windows ① CRITICAL_SECTION — 한 프로세스 안, 유저 모드라 빠르다
InitializeCriticalSection → EnterCriticalSection / LeaveCriticalSection

Windows ② Named Mutex — 프로세스 사이, 이름으로 커널이 찾아 준다
CreateMutex(이름) → WaitForSingleObject / ReleaseMutex

특징

- 상태가 잠김과 풀림 두 가지뿐이다
- 소유권이 있다 — 잠근 스레드만 풀 수 있다
- 락을 못 잡으면 커널이 재워 두었다가 풀릴 때 깨운다 (CPU 를 쓰지 않는다)
- 한 프로세스 안이면 CRITICAL_SECTION, 프로세스 사이면 이름 붙인 Mutex 를 쓴다

어떤 상황에서 쓰는가

- 여러 스레드가 같은 변수나 자료구조를 고칠 때
- 링버퍼의 head · tail 인덱스 갱신, 링크드 리스트 · 해시 테이블

장점

- 쓰임이 단순해서 실수가 적고 디버깅이 쉽다
- 누가 잡고 있는지 분명해 데드락을 추적할 수 있다

단점

- 잡은 쪽만 풀 수 있어 신호 전달에는 못 쓴다
- 락 잡는 순서가 엇갈리면 데드락이 난다
- 조건을 기다리는 데 쓰면 잠갔다 풀었다 확인을 반복해 CPU 를 태운다

발표 대본

2-3 ① 뮉텍스 · 약 1 분 30 초

이제 동기화입니다. 먼저 뮉텍스입니다. 이름 그대로 상호배제, MUTual EXclusion 입니다. 두 스레드가 같은 데이터를 동시에 고치면 값이 깨집니다. count++ 가 그렇습니다. 한 줄 같지만 읽고, 더하고, 쓰는 세 단계라서 둘이 겹치면 두 번 더했는데 결과는 한 번만 늘어 있습니다. 이렇게 겹치면 깨지는 코드 구간을 임계구역이라고 합니다.

뮉텍스는 그 구간에 한 번에 한 명만 들어가게 하는 자물쇠입니다. 들어가기 전에 잠그고, 나오면서 풀립니다. 그림처럼 스레드 1 이 먼저 잡으면 스레드 2 는 풀릴 때까지 기다립니다.

특징은 넷입니다. 상태가 잠김과 풀림 둘뿐이고, 소유권이 있어서 잠근 쪽만 풀 수 있습니다. 락을 못 잡으면 커널이 재워 두었다가 풀릴 때 깨워 줍니다. 그리고 Windows 는 한 프로세스 안이면 CRITICAL_SECTION, 프로세스 사이면 이름 붙인 뮉텍스로 걸립니다. 본 과제도 ToDo#1 은 앞쪽, ToDo#2 는 뒤쪽입니다. 아래 코드의 lock 과 unlock 은 개념이고 실제 이름은 그 아래 API 에 있습니다.

쓰는 곳은 여러 스레드가 같은 변수나 자료구조를 고칠 때입니다. 링버퍼의 head 와 tail 갱신이 바로 그 자리입니다.

장점은 쓰기가 단순해서 실수가 적고, 누가 쥐고 있는지 분명해서 문제가 생겼을 때 추적이 된다는 것입니다.

단점은 셋입니다. 잠근 쪽만 풀 수 있어서 신호 전달에는 쓸 수 없습니다. 그래서 데이터가 올 때까지 기다리는 일을 뮉텍스로 하려고 하면 잠갔다 풀었다 확인을 반복하게 되고, 그때 CPU 를 태웁니다. 이것을 바쁜 대기라고 합니다. 이것이 다음 장 세마포어가 필요한 이유입니다. 마지막으로 락 잡는 순서가 엇갈리면 서로를 기다리며 둘 다 멈추는 데드락이 나는데, 이건 14 장에서 다시 보겠습니다.

예상 질문

CRITICAL_SECTION 이 뭐가 — Windows 의 유저 모드 잠금 객체. 한 프로세스 안의 스레드끼리만 통하고, 그래서 빠르다.

Named Mutex 는 뭐가 다른가 — 커널 오브젝트다. 이름만 같으면 다른 프로세스도 같은 걸 잡는다. 대신 느리다.

lock(m) 이 실제 함수인가 — 아니다. Enter / LeaveCriticalSection, 또는 WaitForSingleObject / ReleaseMutex 다.

왜 신호 전달에는 못 쓰나 — 잠근 쪽만 풀 수 있어서다. 남이 대신 풀어 줄 수 없으니 알림이 안 된다.

뮉텍스도 바쁜 대기인가 — 아니다. 락을 못 잡으면 커널이 재워 준다. 바쁜 대기는 뮉텍스로 조건을 기다리려 할 때만 생긴다.

2-3 세마포어 Semaphore

쓸 수 있는 자원이 몇 개 남았는지 세는 카운터. wait 로 하나 가져가고 post 로 하나 돌려준다

카운터가 자원의 개수를 센다

count = 3

wait (P)

하나 가져간다
count = 0 이면 대기

post (V)

하나 돌려준다
대기 중인 쪽을 깨움

자원 슬롯

사용 가능

사용 가능

사용 가능

사용 중

사용 중

```
sem_wait(&sem); /* 자원 사용 구간 */ sem_post(&sem);
```

API

```
Linux : sem_init / sem_wait / sem_post (POSIX)
       semget / semop (System V)
Windows : CreateSemaphore( 이름 ) 으로 만들고
         WaitForSingleObject / ReleaseSemaphore
```

특징

- 0 이상의 정수 카운터. wait(P) 는 1 감소하고 0 이면 대기, post(V) 는 1 증가하고 대기자를 깨운다 (P · V 는 네덜란드어 검사 · 증가의 첫 글자)
- 소유권이 없어서 A 가 내리고 B 가 올려도 된다
- 0 이면 커널이 재워 두었다가 post 때 깨운다 (기다리는 동안 CPU 를 쓰지 않는다)

어떤 상황에서 쓰는가

- 커넥션 풀이나 버퍼 슬롯처럼 개수가 정해진 자원
- 한쪽이 채우고 다른 쪽이 꺼낼 때 찻다 비었다를 알림

장점

- 개수를 셀 수 있다 (뮤텍스로는 안 된다)
- 조건을 기다리는 일을 확인 반복 없이 할 수 있다
- 소유권이 없어 다른 주체가 신호를 줄 수 있다
- 이름을 주면 프로세스 간에도 쓸 수 있다

단점

- 소유자가 없어 잘못 쓴 지점을 찾기 어렵다
- post 를 한 번 빠뜨리면 영원히 기다린다

발표 대본

2-3 ② 세마포어 · 약 1분 30초

앞 장에서 뮤텍스로는 못 한다고 한 기다리기를 맡는 도구가 세마포어입니다. 쓸 수 있는 자원이 몇 개 남았는지 세는 카운터이고, 동작은 둘뿐입니다. wait 는 하나 가져가면서 카운터를 1 줄이고, 0 이면 남을 때까지 기다립니다. post 는 하나 돌려주면서 1 늘리고, 기다리던 쪽을 깨웁니다. 교과서에서 P 연산, V 연산이라고 부르는 것이 이 둘이고, 네덜란드어 검사와 증가의 첫 글자입니다. 그림처럼 슬롯 다섯 개 중 세 개가 남아 있으면 카운터가 3 입니다.

뮤텍스와 정반대인 점이 하나 있습니다. 소유권이 없습니다. A 가 내리고 B 가 올려도 됩니다. 그래서 다 됐다고 알려 주는 신호로 쓸 수 있습니다. 뮤텍스로는 안 되는 일입니다. 카운터가 0 이면 커널이 재워 줬다가 post 때 깨워 줍니다. 뮤텍스도 락을 기다릴 때는 똑같이 재워 줍니다. 다른 점은 뮤텍스로는 조건 자체를 기다릴 수 없다는 것입니다.

쓰는 곳은 커넥션 풀이나 버퍼 슬롯처럼 개수가 정해진 자원, 그리고 한쪽이 채우고 다른 쪽이 꺼낼 때 찻다 비었다를 알리는 자리입니다. 두 번째가 본 과제의 링버퍼입니다.

장점 네 가지는 방금 말씀드린 것들입니다. 개수를 셀 수 있고, 조건을 기다리는 일을 확인 반복 없이 할 수 있고, 소유권이 없어 남이 신호를 줄 수 있고, 이름을 주면 프로세스 사이에서도 쓸 수 있습니다. 마지막이 ToDo#2 입니다. 단점은 소유자가 없어서 잘못 쓴 지점을 찾기 어렵고, post 를 한 번 빠뜨리면 영원히 기다린다는 것입니다.

아래 코드에서 wait 과 post 사이를 임계구역이라 하지 않고 자원 사용 구간이라고 적었습니다. 카운터가 3 이면 세 쓰레드가 동시에 그 안에 있기 때문입니다. 1 로 두면 그때는 뮤텍스처럼 한 명만 들어갑니다. 함수 이름은 리눅스가 sem_wait 과 sem_post, Windows 가 WaitForSingleObject 와 ReleaseSemaphore 이고, 리눅스에는 System V 계열도 따로 있습니다.

예상 질문

wait 과 post 사이는 임계구역인가 — 아니다. 카운터가 3 이면 세 쓰레드가 동시에 그 안에 있다. 1 로 두면 그때는 임계구역이 된다.
post 를 먼저 불러도 되나 — 된다. 카운터가 먼저 올라가고 나중에 온 wait 이 그냥 통과한다. 순서가 자유로운 점이 뮤텍스와 다르다.
본 과제는 몇 개 쓰나 — 두 개. 빈 슬롯 개수와 찬 슬롯 개수. ToDo#1 은 이름 없이, ToDo#2 는 이름 붙여 프로세스 간에.

2-3 뮤텍스 vs 세마포어 — 왜 둘 다 쓰는가

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 반드시 문제가 생긴다

구분	뮤텍스	세마포어
값	잠김 / 풀림 (0 · 1)	0 이상의 정수 카운터
소유권	있다 — 잠금 풀면 해제	없다 — 누구나 post 가능
목적	상호배제 (데이터 보호)	신호 · 개수 관리 (흐름 제어)
던지는 질문	지금 나만 만지고 있나 ?	쓸 수 있는 게 몇 개 남았나 ?
대표 용도	임계구역 보호	송신 - 수신 버퍼, 자원 풀

뮤텍스만 쓰면

버퍼가 빌 때까지 기다리려면 락을 잡았다 풀었다 확인을 반복 → 바쁜 대기로 CPU 를 태운다

세마포어만 쓰면

개수는 맞지만 head · tail 을 두 쓰레드가 동시에 고쳐 데이터가 깨진다

둘을 같이 쓰면

세마포어로 기다리고, 뮤텍스로 보호한다 → 바쁜 대기도 없고 데이터도 안전하다

송신 - 수신 표준 순서

송신

sem_wait(empty)

lock(mutex)

버퍼에 write

unlock(mutex)

sem_post(full)

수신

sem_wait(full)

lock(mutex)

버퍼에서 read

unlock(mutex)

sem_post(empty)

순서를 뒤집으면 데드락 — 뮤텍스를 먼저 잡고 세마포어를 기다리면, 상대는 그 락을 영원히 잡지 못한다.

발표 대본

2-3 ③ 뮤텍스 vs 세마포어 · 약 2 분

둘을 나누는 기준은 하나입니다. 무엇을 물어보느냐입니다.

뮤텍스는 지금 나만 이 데이터를 만지고 있나를 묻습니다. 데이터를 지키는 도구입니다. 세마포어는 쓸 수 있는 게 몇 개 남았나를 묻습니다. 개수를 세고 기다리는 도구입니다.

표의 나머지도 여기서 같립니다. 값은 뮤텍스가 잠김과 풀림 둘, 세마포어가 0 이상의 카운터입니다. 소유권은 뮤텍스만 있습니다. 그래서 대표 용도도 뮤텍스는 임계구역 보호, 세마포어는 송신 - 수신 버퍼와 자원 풀로 같립니다.

그래서 하나로 다른 하나를 대신할 수 없습니다. 뮤텍스만 쓰면 버퍼가 빌 때까지 기다리는 일을 락을 잡았다 풀었다 반복하는 수밖에 없어서 CPU 를 태웁니다. 세마포어만 쓰면 개수는 맞지만 두 쓰레드가 head 와 tail 을 동시에 고쳐 데이터가 깨집니다. 그래서 둘을 같이 씁니다. 세마포어로 기다리고, 뮤텍스로 보호합니다.

아래가 그 표준 순서입니다. 보내는 쪽은 빈 슬롯 세마포어를 기다리고, 뮤텍스를 잡고, 쓰고, 풀고, 찬 슬롯 세마포어를 올립니다. 받는 쪽은 정확히 반대입니다.

순서가 중요합니다. 반드시 세마포어가 먼저, 뮤텍스가 나중입니다. 뮤텍스를 먼저 잡은 채로 세마포어를 기다리면, 상대는 그 락을 못 잡아서 세마포어를 영원히 올려 주지 못합니다. 둘 다 멈춥니다.

예상 질문

한 줄로 구분하면 — 데이터를 지키면 뮤텍스, 개수를 세거나 알리면 세마포어.

왜 순서가 세마포어 먼저인가 — 기다리는 동안 락을 쥐고 있으면 안 되기 때문이다. 쥔 채로 자면 아무도 깨워 줄 수 없다.

뮤텍스 대신 바이너리 세마포어를 쓰면 — 동작은 하지만 소유권이 없어 남이 풀어도 막지 못한다. 상호배제는 뮤텍스가 맞다.

본 과제도 이 순서인가 — 그렇다. ToDo#1 · ToDo#2 의 송수신 함수가 모두 이 다섯 단계 순서다.

03

데이터 송 / 수신 기능 구현 및 코드 정리

개발 환경

Windows · Visual Studio 2022 (v143) · x64
 개발언어 C — IPC 코어는 전부 C DLL (Dynamic Link Library), 확인용 UI 만 MFC

공통 데이터 시나리오 (세 ToDo 모두 동일)

송신측 1 로 초기화된 int 를 0.1 초 간격으로 송신 , 이후 1 씩 증가 (100 까지)
 수신측 받은 데이터를 그대로 출력

ToDo#1

쓰레드 간 메시지 송 / 수신

전역변수 + 뮤텍스 / 세마포어

IpThread.c

ToDo#2

프로세스 간 메시지 송 / 수신

메시지 큐 (공유메모리 + 링버퍼)

IpMsgQ.c

ToDo#3

다른 프로그램과 메시지 송 / 수신

TCP / IP (SocketUtility 포팅)

IpSocket.c

IPC 로직은 전부 IpCore.dll 안에 있고 , MFC (Microsoft Foundation Class) UI (User Interface) 는 버튼으로 코어 함수를 호출하고 로그만 표시한다 .

발표 대본

섹션 3 · 약 1 분

이제 구현입니다 .

개발 환경은 과제 조건대로 Windows 에서 Visual Studio 2022, x64 구성이고 , 개발언어는 C 입니다 . IPC 기능은 전부 C 로 작성한 DLL 안에 있고 , MFC 는 버튼으로 코어 함수를 호출하고 로그만 표시하는 확인용 UI 입니다 . IPC 로직은 UI 에 한 줄도 두지 않았습니다 .

데이터 시나리오는 세 가지 ToDo 가 모두 동일합니다 . 송신측은 1 로 초기화된 int 를 0.1 초 간격으로 보내고 보낸 뒤 1 씩 증가시켜 100 까지 갑니다 . 수신측은 받은 데이터를 그대로 출력합니다 . 같은 시나리오를 쓰기 때문에 세 방식의 결과를 그대로 비교할 수 있습니다 .

ToDo#1 은 쓰레드 간 송수신을 전역변수와 뮤텍스 · 세마포어로 , ToDo#2 는 프로세스 간 송수신을 메시지 큐로 , ToDo#3 은 다른 프로그램과의 송수신을 TCP/IP 로 구현했고 각각 IpThread.c, IpMsgQ.c, IpSocket.c 파일에 있습니다 .

ToDo#1 쓰레드 간 송 / 수신 구현

전역 링버퍼를 뮤텍스로 보호하고, 빈 슬롯 · 찬 슬롯 개수를 세마포어 2 개로 센다 · ipcThread.c



LINUX 대응

CRITICAL_SECTION → pthread_mutex_t
 CreateSemaphore / WaitForSingleObject → sem_init / sem_wait
 ReleaseSemaphore → sem_post
 _beginthreadex → pthread_create
 * CreateThread 대신 _beginthreadex — CRT (C Runtime) 자원이 정리되도록

```
f_IpcThreadQueueSend()
// ① 빈 슬롯 대기 — 가득 차면 여기서 블록
if (WaitForSingleObject(s_hSemEmpty, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 뮤텍스로 링버퍼 상호배제
EnterCriticalSection(&s_stRingLock);
s_staRing[s_iRingTail] = *stpMsg;
s_iRingTail = (s_iRingTail + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount++;
LeaveCriticalSection(&s_stRingLock);

// ③ 찬 슬롯 +1 — 수신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemFull, 1, NULL);
```

```
f_IpcThreadQueueRecv()
// ① 찬 슬롯 대기 — 비어 있으면 여기서 블록
if (WaitForSingleObject(s_hSemFull, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 같은 뮤텍스로 상호배제
EnterCriticalSection(&s_stRingLock);
*stpMsg = s_staRing[s_iRingHead];
s_iRingHead = (s_iRingHead + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount--;
LeaveCriticalSection(&s_stRingLock);

// ③ 빈 슬롯 +1 — 송신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemEmpty, 1, NULL);
```

발표 대본

ToDo#1 구현 · 약 2 분

ToDo#1 은 쓰레드 간 송수신입니다. 과제 조건이 전역변수에 뮤텍스와 세마포어를 쓰는 것이었습니다.

왼쪽 구조를 보시면, 송신 쓰레드와 수신 쓰레드 사이에 전역 링버퍼 8 슬롯을 두었습니다. 그리고 동기화 객체가 세 개입니다. 뮤텍스 역할의 CRITICAL_SECTION 하나로 head 와 tail 을 보호하고, 세마포어 두 개로 빈 슬롯과 찬 슬롯의 개수를 셉니다. 빈 슬롯 세마포어는 초기값 8, 찬 슬롯 세마포어는 초기값 0 으로 시작합니다.

오른쪽이 실제 코드입니다. 송신은 세 단계입니다. 먼저 빈 슬롯 세마포어를 기다립니다. 버퍼가 가득 차 있으면 여기서 블록됩니다. 그다음 뮤텍스를 잡고 링버퍼에 쓰고 tail 을 하나 돌립니다. 마지막으로 찬 슬롯 세마포어를 올려서 수신 쓰레드를 깨웁니다.

수신은 정확히 대칭입니다. 찬 슬롯을 기다리고, 같은 뮤텍스로 보호하고, 읽고, 빈 슬롯을 올립니다. 앞에서 본 송신 - 수신 표준 순서 그대로입니다.

왼쪽 아래는 리눅스 대응입니다. CRITICAL_SECTION 은 pthread_mutex_t, CreateSemaphore 와 WaitForSingleObject 는 sem_init 과 sem_wait, ReleaseSemaphore 는 sem_post, _beginthreadex 는 pthread_create 에 대응합니다. 참고로 쓰레드 생성에 CreateThread 가 아니라 _beginthreadex 를 쓴 이유는, CRT 를 쓰는 쓰레드는 이쪽으로 만들어야 종료할 때 CRT 자원이 정리되기 때문입니다.

ToDo#1

실행 결과

한 프로세스 안에서 [Start] — 송신과 수신이 0.1 초 간격으로 교대로 찍힌다

```

● IpcUI — 로그창 · Thread [Start] → [Stop]

14:20:31 [TH] start
14:20:31 [TH] tx 1
14:20:31 [TH] rx 1
14:20:31 [TH] tx 2
14:20:31 [TH] rx 2
14:20:31 [TH] tx 3
14:20:31 [TH] rx 3
14:20:32 [TH] tx 4
14:20:32 [TH] rx 4
14:20:32 [TH] tx 5
14:20:32 [TH] rx 5
...
14:20:41 [TH] tx 98
14:20:41 [TH] rx 98
14:20:41 [TH] tx 99
14:20:41 [TH] rx 99
14:20:41 [TH] tx 100
14:20:41 [TH] rx 100
14:20:41 [TH] tx done (100)
14:20:46 [TH] rx done (100, seq err 0)
14:20:46 [TH] stop

```

무엇을 확인했나

1 1 부터 100 까지 빠짐없이

송신 100 건, 수신 100 건. 순서가 뒤바뀌거나 빠진 값이 없다.

2 tx 와 rx 가 교대로

링버퍼가 8 칸뿐이라 수신이 밀리면 송신이 세마포어에서 대기한다 — 흐름 제어가 동작한다는 뜻이다.

3 대기 중에도 [Stop] 에 즉시 반응

수신 스레드는 100ms 단위로 타임아웃을 두고 대기하므로 정지 요청이 바로 먹힌다.

실행 방법

IpcUI.exe 를 실행하고 Thread 영역의 [Start] 버튼 하나만 누르면 된다.
한 프로세스 안에서 송신 스레드와 수신 스레드가 한 쌍으로 뜬다.

발표 대본

ToDo#1 실행 결과 · 약 1 분

실행 결과입니다. IpcUI 를 띄우고 Thread 영역의 [Start] 버튼 하나만 누르면 한 프로세스 안에서 송신 스레드와 수신 스레드가 한 쌍으로 뜬다.

로그를 보시면 start 이후 tx 1, rx 1, tx 2, rx 2 순서로 0.1 초 간격으로 찍히고, 100 까지 간 뒤 tx done (100) 이 나옵니다. 마지막 줄의 seq err 0 은 수신이 순번을 하나하나 대조한 결과로, 송신 100 건이 순서대로 빠짐없이 도착했다는 뜻입니다.

두 번째로 볼 것은 tx 와 rx 가 교대로 찍힌다는 점입니다. 다만 교대는 세마포어가 보장하는 것이 아니라 0.1 초 간격 덕분입니다. 송신이 한 건 보내고 쉬는 동안 수신이 다 꺼내 가서 큐가 매번 비기 때문입니다. 세마포어가 보장하는 것은 순서와 유실 없음이고, 링버퍼가 8 칸이라 수신이 밀리면 송신이 빈 슬롯 세마포어에서 대기합니다. 흐름 제어는 그때 동작합니다.

세 번째는 정지 반응입니다. 송신은 100 까지 보내고 스스로 끝나지만 수신 스레드는 계속 대기합니다. 그런데 수신은 100 밀리초 단위로 타임아웃을 두고 기다리기 때문에, [Stop] 을 누르면 대기 중이어도 바로 빠져나와 rx done 을 찍고 종료합니다.

예상 질문

세마포어를 쓰면 무조건 번갈아 나오나 — 아니다. 교대는 0.1 초 간격 덕분이다. 세마포어가 보장하는 것은 순서와 유실 없음이다.

간격을 없애면 — 송신이 슬롯 8 개를 다 채울 때까지 앞서가서 tx 가 몰려 나온 뒤 rx 가 몰려 나온다. 그래도 정상이다.

로그 줄 순서는 믿을 수 있나 — 있다. 송신은 상대를 깨우기 전에, 수신은 빈 슬롯을 돌려주기 전에 찍는다.

seq err 0 이 무슨 뜻인가 — 수신이 기대 순번과 실제 순번을 대조한 결과다. 0 이면 순서가 한 번도 안 어긋났다는 뜻이다.

ToDo#2 메시지 큐 구현

리눅스 환경의 메시지 큐가 윈도우 환경에는 없어서 공유메모리 + 링버퍼로 메시지 큐 구현 · lpcMsgQ.c

리눅스 메시지 큐 API ↔ 이 과제의 Windows 구현

Linux (System V) 본 과제의 Windows 구현

<code>msgget(key, IPC_CREAT)</code>	CreateFileMapping — 이름 붙인 공유메모리 + 링버퍼 헤더 초기화
<code>msgsnd()</code>	세마포어 (빈 슬롯) → 뮤텍스 → write → 세마포어 (찬 슬롯) +1
<code>msgrcv()</code>	세마포어 (찬 슬롯) → 뮤텍스 → read → 세마포어 (빈 슬롯) +1
<code>msgctl(IPC_RMID)</code>	모든 핸들 CloseHandle — 마지막 참조가 사라지면 정리된다
<code>mtype</code>	ST_IpcMsg, iMsgType — 메시지 종류를 구분하는 필드

※ 이름 하나로 같은 큐를 찾아 보고, 가득 차면 보내는 쪽이 · 비어 있으면 받는 쪽이 대기한다

큐 하나 = 공유메모리 + 링버퍼 + 세마포어 2 개 + 뮤텍스

두 프로세스는 서로의 메모리를 못 본다 — 같은 이름을 커널에 대면 같은 물건을 받는다

이름은 모두 Local\IpcProc.IpcDemoQ 로 시작하고 뒤에 넷이 붙는다

<code>.map</code>	공유메모리 — 링버퍼 64 슬롯이 여기 올라간다
<code>.mtx</code>	뮤텍스 — head, tail 을 한 번에 하나만
<code>.sem.e</code>	세마포어 — 빈 슬롯 수 (처음 64)
<code>.sem.f</code>	세마포어 — 찬 슬롯 수 (처음 0)

※ Local\ 이라 관리자 권한이 필요 없다 · Create 는 없으면 만들고 있으면 참여 → 기동 순서 무관

```
f_IpcMsgQSend() == msgsnd()
// ① 빈 슬롯 대기 (프로세스 간 세마포어)
WaitForSingleObject(stpQ->vpSemEmpty, uiTimeout_ms);

// ② 프로세스 간 상호배제 — 네임드 뮤텍스
WaitForSingleObject(stpQ->vpMutex, 50000);

// ③ 공유메모리 위의 링버퍼에 write
stpRing->staSlot[stpRing->iTail] = *stpMsg;
stpRing->iTail = (stpRing->iTail + 1) % stpRing->iCapacity;
stpRing->nCount++;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemFull, 1, NULL); // ④ 찬 슬롯 +1
```

```
f_IpcMsgQRecv() == msgrcv()
// ① 찬 슬롯 대기
WaitForSingleObject(stpQ->vpSemFull, uiTimeout_ms);

// ② 같은 네임드 뮤텍스
WaitForSingleObject(stpQ->vpMutex, 50000);

// ③ 링버퍼에서 read
*stpMsg = stpRing->staSlot[stpRing->iHead];
stpRing->iHead = (stpRing->iHead + 1) % stpRing->iCapacity;
stpRing->nCount--;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemEmpty, 1, NULL); // ④ 빈 슬롯 +1
```

발표 대본

ToDo#2 구현 · 약 2 분 30 초

ToDo#2 는 프로세스 간 송수신입니다. 과제는 메시지 큐로 구현하라고 되어 있는데, 개발 환경인 Windows 에는 System V 메시지 큐가 없습니다. msgget, msgsnd, msgrcv, msgctl 이 아예 존재하지 않습니다. 그래서 앞에서 본 조합으로 같은 동작을 직접 만들었습니다.

왼쪽 위가 그 대응 관계입니다. msgget 은 CreateFileMapping 으로 이름 붙인 공유메모리를 만들고 링버퍼 헤더를 초기화하는 것으로, msgsnd 는 빈 슬롯 세마포어를 기다리고 뮤텍스를 잡고 링버퍼에 쓰고 찬 슬롯 세마포어를 올리는 순서로 대응합니다. msgrcv 는 그 반대입니다. IPC_RMID 는 핸들을 모두 닫는 것으로, mtype 은 iMsgType 필드로 대응했습니다.

왼쪽 아래가 실제 구성입니다. 큐 하나는 공유메모리, 링버퍼, 세마포어 두 개, 뮤텍스가 합쳐진 것입니다. 여기서 커널 오브젝트라는 말이 나오는데, 커널이 들고 있는 자원에 붙인 이름표라고 보시면 됩니다.

두 프로세스는 서로의 메모리를 볼 수 없습니다. 그래서 같은 이름을 커널에 대면 커널이 같은 물건을 쥐어 줍니다. 리눅스에서 ftok 으로 만든 key 와 같은 역할입니다. 이름은 모두 Local\IpcProc.IpcDemoQ 로 시작하고 뒤에 map, mtx, sem.e, sem.f 넷이 붙습니다.

Global 이 아니라 Local 을 쓴 것은 관리자 권한 없이 실행하기 위해서입니다. 그리고 Create 를 없으면 만들고 있으면 참여한다로 만들어서, [Recv] 를 먼저 눌러도 [Send] 를 먼저 눌러도 동작합니다.

오른쪽이 실제 송수신 코드입니다. 앞에서 본 다섯 단계와 완전히 같습니다. 세마포어 대기, 뮤텍스, 링버퍼 접근, 뮤텍스 해제, 반대쪽 세마포어 증가입니다. 다른 점은 프로세스 간 커널 오브젝트를 쓴다는 것뿐이고, 결국 msgsnd 와 msgrcv 를 이 코드가 대신하고 있습니다.

예상 질문

커널 오브젝트가 뭐가 — 커널이 들고 있는 자원 (공유메모리 · 뮤텍스 · 세마포어) 에 붙인 이름표다. 같은 이름을 대면 커널이 같은 물건을 준다. 리눅스의 ftok key 와 같은 역할이다.

세마포어 초기값이 두 번 반영되지 않나 — CreateSemaphore 의 초기 카운트는 최초 생성 때만 먹는다. 두 번째로 붙는 프로세스는 기존 것을 열 뿐이라 덮어쓰지 않는다.

ToDo#2

실행 결과

[New Process] 로 창을 하나 더 띄우고, 한쪽은 [Recv] · 다른쪽은 [Send] 를 누른다

● 프로세스 ① 수신 — [Recv] 를 먼저 눌러 큐를 만든다

```
14:22:05 [MQ] queue 'IpcDemoQ' open
14:22:05 [MQ] start (rx)
14:22:12 [MQ] rx 1
14:22:12 [MQ] rx 2
14:22:12 [MQ] rx 3
14:22:12 [MQ] rx 4
14:22:12 [MQ] rx 5
14:22:13 [MQ] rx 6
...
14:22:22 [MQ] rx 98
14:22:22 [MQ] rx 99
14:22:22 [MQ] rx 100
14:22:28 [MQ] rx done (100)
14:22:28 [MQ] stop
```

● 프로세스 ② 송신 — [New Process] 로 띄운 창에서 [Send]

```
14:22:12 [MQ] queue 'IpcDemoQ' open
14:22:12 [MQ] start (tx)
14:22:12 [MQ] tx 1
14:22:12 [MQ] tx 2
14:22:12 [MQ] tx 3
14:22:12 [MQ] tx 4
14:22:12 [MQ] tx 5
14:22:13 [MQ] tx 6
...
14:22:22 [MQ] tx 98
14:22:22 [MQ] tx 99
14:22:22 [MQ] tx 100
14:22:22 [MQ] tx done (100)
14:22:26 [MQ] stop
```

1

두 프로세스 사이로 100 건이 그대로 전달

송신 창의 tx 1~100 과 수신 창의 rx 1~100 이 정확히 일치한다.

2

수신을 먼저 눌러도, 송신을 먼저 눌러도 동작

Create 가 없으면 만들고 있으면 참여하므로 기동 순서에 의존하지 않는다.

3

수신 창의 시각이 7 초 늦게 시작

큐가 먼저 만들어져 대기하고 있다가, 송신이 붙는 순간부터 받기 시작한다 — 비동기 동작 확인.

발표 대본

ToDo#2 실행 결과 · 약 1 분 30 초

실행 결과입니다. 확인 방법은 [New Process] 버튼으로 같은 프로그램 창을 하나 더 띄우고, 한쪽에서 [Recv], 다른쪽에서 [Send] 를 누르는 것입니다.

왼쪽이 수신 프로세스입니다. 22 분 05 초에 queue 'IpcDemoQ' open 이 찍히고 rx 로 시작합니다. 오른쪽 송신 프로세스는 7 초 뒤인 22 분 12 초에 같은 큐에 붙어서 tx 1 부터 보내기 시작합니다.

여기서 세 가지를 확인할 수 있습니다.

첫째, 송신 창의 tx 1 부터 100 까지와 수신 창의 rx 1 부터 100 까지 정확히 일치합니다. 두 프로세스 사이로 100 건이 그대로 전달됐습니다.

둘째, 수신을 먼저 눌러도 송신을 먼저 눌러도 동작합니다. Create 가 없으면 만들고 있으면 참여하기 때문에 기동 순서에 의존하지 않습니다.

셋째, 시각을 보시면 수신 쪽이 7 초 먼저 떠서 큐를 만들어 놓고 대기하다가, 송신이 붙는 순간부터 받기 시작했습니다. 메시지 큐의 비동기 성질이 그대로 재현된 것입니다.

ToDo#3 TCP / IP 구현

다른 PC · 다른 프로그램과의 송수신 — Linux SocketUtility v5.0 → Winsock2 포팅, 함수명 · 인자 · 상수는 원본 그대로 · lpcSocket.c

연결 맺는 순서 — 문을 열고 기다리는 쪽이 서버, 찾아가서 두드리는 쪽이 클라이언트

① 먼저 Rx · 수신 = 서버 [Recv] — 자리를 잡고 기다린다



② 그다음 Tx · 송신 = 클라이언트 [Send] — 그 자리를 찾아간다



※ accept 는 200ms 단위 select 루프, connect 는 논블로킹 + select — [Stop] 에 바로 반응한다

리눅스 원본을 Windows 로 옮기며 걸린 것 — 컴파일은 통과하는데 뜻이 달라지는 항목

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) · x64 에서 64bit
송 / 수신 타임아웃	timeval 구조체	DWORD 밀리초 — 그대로 넣기면 엉뚱한 값
소켓 닫기	close()	closesocket()
초기화	없음	WSAStartup / WSACleanup 필요
에러 확인	errno	WSAGetLastError()
select 1 번 인자	maxfd + 1	무시된다 (있어도 되고 없어도 된다)

※ 타임아웃은 형이 맞아 보여 그냥 지나치기 쉽다 — 내부에서 timeval → DWORD ms 로 변환했다

f_SocketRecvTCP_IPv4() — TCP 는 바이트 스트림, 다 채울 때까지 반복

send(4 byte) 를 한 번 불러도 recv 는 2 byte 만 받을 수 있다 — 메시지 경계가 없다

```
// ① 요청한 크기를 다 채울 때까지 반복해서 받는다
while ((lRemain > 0) && (stpSocket->iSockStatus == Connected)) {
    iRecv = recv((SOCKET)stpSocket->hSockId, cpCursor, iChunk, 0);

    if (iRecv > 0) {
        // ② 받은 만큼 커서를 민다
        cpCursor += iRecv;
        lTotalRecv += iRecv;
        lRemain = lFixedDataSize - lTotalRecv;
    }
    else stpSocket->iSockStatus = Disconnected; // ③ 상태 종료 · 에러
}
```

f_TcpSenderProc() — 데모 송신, 프레임 헤더 없이 int 4 byte

```
// 1 부터 100 까지 0.1 초 간격으로 보낸다
for (iData = IPC_DEMO_FIRST_VALUE;
    iData <= IPC_DEMO_LAST_VALUE; iData++) {
    // htonl 체크박스가 켜져 있으면 네트워크 바이트 오더로 실행한다
    iWire = (s_iDemoUseNB0 != 0) ? htonl(iData) : iData;
    lSent = f_SocketSendTCP_IPv4(&s_stDemoSocket, &iWire, sizeof(iWire));
    // 0.1 초 쉬면서 정지 요청도 같이 확인한다
    f_IsDemoStopRequested(IPC_DEMO_INTERVAL_MS);
}
```

경계는 없이 정한다 — ① 길이 붙이기 ② 구분자 ③ 고정 길이 중 본 과제는 ③ 번 양쪽 다 little-endian 이라 그대로 맞고, 상대가 htonl 규약이면 체크박스를 켜다 보내는 쪽 f_SocketSendTCP_IPv4() 도 같은 모양 — 다 보낼 때까지 반복한다

발표 대본

ToDo#3 구현 · 약 2 분

ToDo#3 은 다른 PC, 다른 프로그램과 주고받는 것이고 TCP/IP 로 구현했습니다. 첨부받은 Linux SocketUtility 를 Winsock2 로 포팅했고, 상대 코드와 맞물려야 하니 함수명과 인자, 상수는 원본 그대로 두었습니다.

왼쪽 위가 연결을 맺는 순서입니다. 소켓은 전화와 비슷해서, 한쪽이 번호를 열어 두고 기다려야 다른 쪽이 겁니다. 기다리는 쪽이 서버, 거는 쪽이 클라이언트입니다. 그래서 수신 Rx 가 서버로 socket, bind, listen, accept 로 먼저 자리를 잡고, 송신 Tx 가 클라이언트로 connect 해서 찾아갑니다. UI 에서 [Recv] 를 먼저 누르는 이유입니다.

accept 와 connect 는 상대를 기다리는 함수라 그냥 부르면 정지 버튼이 먹지 않습니다. 그래서 accept 는 200 밀리초 단위 select 루프로 나눴고 connect 는 논블로킹에 select 를 붙였습니다.

오른쪽 위가 이번 구현의 핵심입니다. TCP 는 바이트 스트림이라 메시지 경계가 없습니다. 4 바이트를 한 번에 보내도 수신은 2 바이트만 받을 수 있습니다. 그래서 다 채울 때까지 반복해서 받아야 하고, 오른쪽 위 f_SocketRecvTCP_IPv4 의 while 루프가 그 일을 합니다.

오른쪽 아래가 데모 송신 코드입니다. 경계는 길이 붙이기, 구분자, 고정 길이 셋 중 하나인데 저희는 고정 길이입니다. 양쪽 다 little-endian 이라 값이 맞고, htonl 체크박스를 켜면 그 자리에서 바뀌 실행합니다.

왼쪽 아래가 포팅에서 걸렸던 것들입니다. 그중 빨강게 표시한 타임아웃이 가장 위험했습니다. 리눅스는 timeval 구조체를 넘기는데 Windows 는 DWORD 밀리초를 받습니다. 캐스팅이 되니 컴파일도 경고도 통과하는데, 값의 뜻이 달라져 런타임에만 이상해집니다. 그래서 내부에서 변환합니다.

예상 질문

왜 수신 쪽이 서버인가 — 원본 SocketUtility 가 UDP 에서 Tx 는 목적지 주소, Rx 는 자기 bind 주소를 받는 구조여서 TCP 도 같게 맞췄다. 상대가 반대라면 Tx / Rx 의 Init 함수만 바꿔 부르면 된다.

컴파일은 되는데 값이 달라지는 항목 — SO_RCVTIMEO 가 Linux 는 timeval, Windows 는 밀리초 DWORD 다. 소켓 핸들이 int 가 아니라 64 비트 SOCKET 인 것도 같은 종류의 함정이다.

ToDo#3

실행 결과

한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트) — 다른 PC 의 프로그램과도 같은 방식으로 붙는다

프로세스 ① 수신 = 서버 — [Recv]

```
14:25:02 [TCP] start (rx)
14:25:02 [TCP] listen 0.0.0.0:51000
14:25:09 [TCP] accept 127.0.0.1:60312
14:25:09 [TCP] rx 1
14:25:09 [TCP] rx 2
14:25:09 [TCP] rx 3
14:25:10 [TCP] rx 4
14:25:10 [TCP] rx 5
...
14:25:19 [TCP] rx 98
14:25:19 [TCP] rx 99
14:25:19 [TCP] rx 100
14:25:20 [TCP] peer closed
14:25:20 [TCP] rx done (100)
```

프로세스 ② 송신 = 클라이언트 — [Send]

```
14:25:09 [TCP] start (tx)
14:25:09 [TCP] connecting 127.0.0.1:51000
14:25:09 [TCP] connected 127.0.0.1:51000
14:25:09 [TCP] tx 1
14:25:09 [TCP] tx 2
14:25:09 [TCP] tx 3
14:25:10 [TCP] tx 4
14:25:10 [TCP] tx 5
...
14:25:19 [TCP] tx 98
14:25:19 [TCP] tx 99
14:25:19 [TCP] tx 100
14:25:19 [TCP] tx done (100)
14:25:20 [TCP] stop
```

1 연결 → 100 건 → 정상 종료

accept 로 상대 IP · 포트를 확인하고, 100 건을 받은 뒤 상대가 끊으면 peer closed 로 마무리된다.

2 다른 PC 와 붙일 때

수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 연다. 상대가 htonl 을 쓰면 체크박스를 켜다.

3 역할이 반대인 상대와도 가능

Tx / Rx Init 함수를 바꿔 부르지만 하면 된다. 함수명 · 상수는 원본 SocketUtility 와 동일하게 두었다.

발표 대본

ToDo#3 실행 결과 · 약 1 분 30 초

TCP 실행 결과입니다. 마찬가지로 창을 두 개 띄우고 한쪽은 [Recv] 로 서버, 다른쪽은 [Send] 로 클라이언트를 시작합니다.

왼쪽 서버 로그를 보시면 listen 0.0.0.0:51000 으로 대기하다가, 7 초 뒤 클라이언트가 붙으면서 accept 에 상대 IP 와 포트가 찍힙니다. 그리고 rx 1 부터 100 까지 받습니다. 클라이언트가 100 건을 다 보내고 소켓을 닫으면 peer closed 가 뜨고 rx done (100) 으로 마무리됩니다.

오른쪽 클라이언트는 connecting, connected 이후 tx 1 부터 100 까지 보내고 tx done (100) 으로 끝납니다. 양쪽 카운트가 100 으로 일치합니다.

다른 PC 와 붙일 때는 수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 열면 됩니다. 상대가 htonl 을 쓰면 체크박스만 켜면 되고, 상대의 Tx/Rx 역할이 반대라면 Init 함수를 바꿔 부르면 됩니다. 함수명과 상수를 원본 SocketUtility 와 동일하게 유지한 것이 이런 상황에서 도움이 됩니다.

예상 질문

바이트 순서는 괜찮은가 — x64 Windows 와 x86-64 Linux 는 둘 다 리틀 엔디안이라 그대로 값이 맞는다. 상대가 htonl 규약이면 체크박스를 켜서 맞춘다.

UDP 를 구현해 놓고 왜 데모엔 안 쓰나 — TCP Sync 라인 체크에 쓴다. 0xAAAA → 0xB BBB → 0xC CCC → 0xD DDD 순서로 원본 SocketUtility v5.0 규약을 그대로 따랐다.

UDP 로 64KB 를 한 번에 보내면 — IP 가 MTU(보통 1500) 단위로 쪼개는데, 조각 하나만 잃어도 데이터그램 전체가 버려진다. 그래서 분할 송신 함수를 따로 두었다.

SIO_UDP_CONNRESET 은 왜 켜나 — 상대 포트가 안 열려 있으면 ICMP 응답 때문에 다음 recvfrom 이 10054 로 깨지는 Windows 동작이라, Linux 와 흐름을 맞추려고 켜다.

포트 51000 의 의미 — 특별한 의미는 없다. 잘 쓰이는 서비스와 겹치지 않는 값으로 고른 것이고 UI 에서 바꿀 수 있다.

정리

개념 세 가지와, 그 개념이 코드에서 어떻게 만났는지

1

뮤텍스와 세마포어는 짝이다

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어.
하나로 다른 하나를 흉내내면 바쁜 대기가 되거나 데이터가 깨진다.

2

메시지 큐는 없으면 만들 수 있다

공유메모리 + 링버퍼 (FIFO) + 세마포어 2 개 + 뮤텍스.
Windows 에 System V 메시지 큐가 없어 이 조합으로 msgsnd / msgrcv 와 같은 의미를 만들었다.

3

포팅에서 위험한 것은 타입이 맞아떨어지는 차이다

SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을
먼저 찾아야 한다. 소켓 핸들 크기, 타입아웃 단위가 대표적이다.

검증 결과 ToDo#1 · #2 · #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고, 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했다.

발표 대본

정리 · 약 1 분

정리하겠습니다. 세 가지를 남기고 싶습니다.

첫째, 뮤텍스와 세마포어는 짝입니다. 상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어입니다. 하나로 다른 하나를 흉내내면 바쁜 대기가 되거나 데이터가 깨집니다. 셋 중 어느 ToDo 를 봐도 이 조합이 그대로 들어가 있습니다.

둘째, 메시지 큐는 없으면 만들 수 있습니다. 같은 메모리를 함께 보는 공유메모리, 넣은 순서대로 꺼내는 링버퍼, 세마포어 두 개, 뮤텍스 하나를 합치면 msgsnd 와 msgrcv 와 같은 의미가 됩니다. Windows 에 System V 메시지 큐가 없다는 제약이 오히려 메시지 큐가 내부적으로 무엇을 하고 있는지 이해하는 계기가 됐습니다.

셋째, 포팅에서 위험한 것은 타입이 맞아 보이는 차이입니다. SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을 먼저 찾아야 합니다.

검증 결과, ToDo#1, #2, #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했습니다.

이상 발표를 마치겠습니다. 감사합니다.